

INF1103: Lab 3

Modular Design

1. Introduction

Welcome to Week 3! Last week, you built a working inventory auditor. It successfully processed input, handled errors, and tracked state inside a loop.

Remember, the focus of this module is all about bridging logical programming constructs with modern, automated operational practices. We will systematically evolve our applications week by week, refactoring and packaging our code to fit enterprise-grade deployment standards.

The expectation is that you complete and verify your lab by the end of your scheduled lab session. True engineering mastery comes from hands-on problem-solving. Ensure you work through and complete each step of this lab by yourself. While your peers are excellent resources for discussing conceptual architectures, they must not write code for you or share their files with you!

It is highly critical that you have watched and engaged with the Week 1 lecture lessons before attempting this lab. We assume you have acquired the foundational mental models—like standard shipping containers, blueprints vs. instances, and environment drift—which you will now put into practice.

Do not hesitate to ask plenty of questions to your professor and teaching assistants when you hit an infrastructure error. We are here to support you as you take your first steps into the DevOps pipeline. Good luck!

However, professional code is rarely written as one giant block. It needs to be modular, reusable, and easy to debug. This week, we will refactor your Week 2 code into functions. You will learn the difference between global state and local scope, and how to pass data between parts of your program using parameters and return values.

Remember:

- Your goal today is refactoring. The code should do exactly what it did last week, but the internal structure must change significantly.
- Functions must be "pure"—they should take input, perform a task, and return a value, rather than relying on global variables.

2. Assignment

1. The Scenario

The store manager is happy with the audit logic, but they want to add new features—like tax calculation and discount summaries—in the coming weeks. They have realized that adding more features to your existing Week 2 loop will make the code impossible to manage. They have tasked you with "Modularizing" the code.

Requirements:

Create a copy of your auditor.py from last week and name it modular_auditor.py. This ensures you have a backup of your Week 2 solution.

1. Initialize the inventory to zero in the start
2. Run in a continuous loop asking user to enter a stock quantity, until the user types quit.
3. If the user enters a valid value:
 - Add the delivery amount to the running total.
 - Calculate the tax for that delivery.
 - Update any counters and records you are tracking, such as the number of deliveries processed.
4. Reporting: When the user types quit, print the *Total Deliveries Processed* and the *Number of Failed/Rejected Entries*.

Your program must now consist of specific functions:

1. `get_valid_input()`: Handles the prompt, handles input validation, and returns a valid integer or a "quit" signal.
2. `process_delivery(current_total, new_value)`: Calculates the new total and returns it.
3. `calculate_tax(amount)`: A new requirement! This function takes a delivery amount and returns the tax (10% of that specific delivery).
4. `generate_report(total_units, failed_attempts)`: A dedicated function to print the final summary.

Don't write code straight away. Map out your function signatures. A function signature defines the inputs and outputs. Example: `def process_delivery(total, value): return new_total`. Map your logic:

- What goes *into* `get_valid_input()`? (Nothing). What comes *out*? (The number or a 'quit' status).

- What goes *into* calculate_tax()? (The delivery value). What comes *out*? (The tax amount).
- Then write the code

2. Initialize Your Workspace & Git

- a. In your terminal, initialize your Git status and add the new file to your repository
- b. Commit regularly as you implement each requirement of the lab.
- c. Push the code to your github repository

3. Dockerize the Run

Run your refactored code using your Docker volume mount command

4. Self-Reflection Task

Consider this for your next module update:

"Why is it better to have a calculate_tax function that simply returns a value, rather than having it print the tax amount directly inside the function?"

(Hint: Think about what happens if the manager suddenly wants the tax sent to a file instead of the screen.)

What you need to submit?

1. Your python file (modular_auditor.py)
2. Your git log. Store it in a text file.
git log >log.txt
3. Docker Mount and run. A screenshot of the successful execution is usually sufficient.
4. Run the following command for evidence that the image was successfully built.
Screenshot of the output.
`docker images`
5. Proof that the image runs successfully with a screenshot showing the program accepting input and producing results.
6. Put everything in a folder. Zip it and upload to xSite.